



King's College London

Department of Engineering

7CCE4EGP & 7CCEMRGP

Group Project Portfolio

Group 15: TIAGo Assistive Robot for Chemotherapy Wards

Team Members: Arjun Sharma
Chengqi Xue
Deema Arsalan
Patricia Lopez
Yixuan Zhuang
Yuchen Wang

Supervisor: Dr. Oya Celiktutan

April 2026

1 Product Brief

1.1 Problem Analysis and Solution Objective

This project develops a socially assistive robotic system based on the TIAGo platform, designed to support patients in chemotherapy wards with low-acuity, non-clinical assistance. The aim is to reduce the burden of small but frequent requests on healthcare staff, while improving patient comfort during long treatment sessions.

Chemotherapy patients may remain seated for extended periods and often need minor support such as requesting water, a blanket, or basic assistance. While these tasks are simple, they can accumulate throughout the day and interrupt clinical workflows. Our proposed system positions TIAGo as a ward-based support robot that can respond to these routine needs in a safe, respectful, and human-centred way. Rather than replacing clinical staff, it is intended to assist with repetitive non-medical tasks so that healthcare professionals can remain focused on higher-priority care. Therefore, patient needs can be attended to even when human staff are not currently available.

Working in partnership with Guy's and St Thomas' Hospital, the primary objective of the project was to develop a socially acceptable robotics system which was appropriate for use in hospital environments. It contributes to a wider adoption of robotics in healthcare environments and their use for personal assistance tasks.

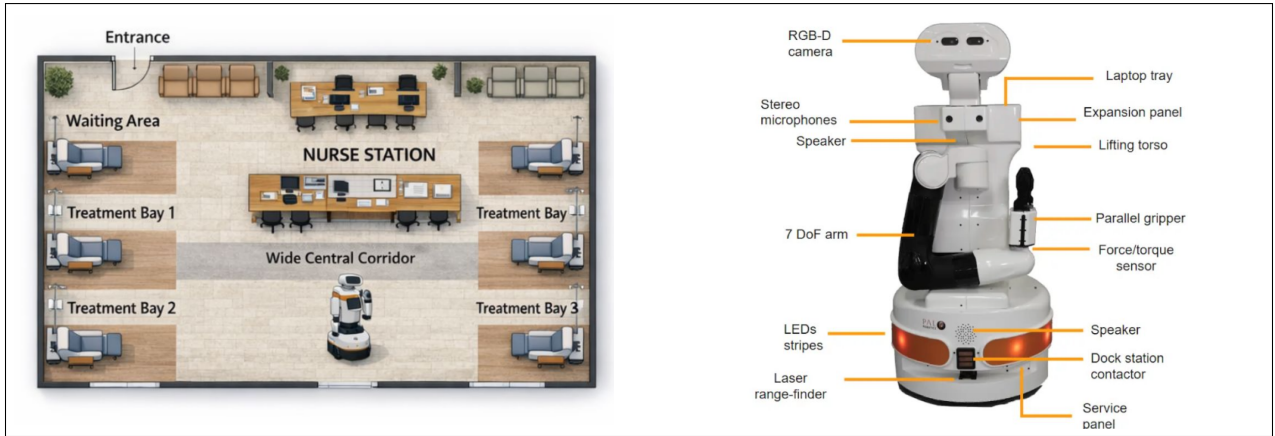


Figure 1: Chemotherapy ward floorplan (left) and the TIAGo robot (right) used as the basis for the assistive care system.

1.2 Product Overview and Features

The finished product consists of a ROS2-based simulation which models the TIAGo robot in a mock hospital environment. The system combines three core functions which the robot performs at a high level:

- **Request detection:** The system identifies when a patient is asking for assistance through a simple engagement-detection stage. This involves detecting a generalised waving gesture combined with a spoken voice command. After confirming both actions, the robot determines that a patient requires help.
- **Safe movement and navigation:** Once a request is recognised, the robot moves towards the patient while maintaining safe and appropriate behaviour within the ward environment. It avoids collisions when moving through mapping its environment and keeps to a socially acceptable distance from the patient during interaction.
- **Predefined response:** After reaching the patient, the robot provides support through a limited set of predefined interaction options.

Using these functions, the robot is able to behave similar to a human staff worker. It is able to perceive when a patient requires help, safely navigate around obstacles towards them and provide a level of

assistance. It repeats this process continuously to provide a useful service to the hospital. A key feature of the product is its emphasis on acceptability in a hospital setting. The robot is intended to operate with clear safety boundaries, maintain an appropriate interaction distance, and support patient dignity and autonomy. Its role is therefore deliberately limited to supportive, low-risk tasks within a clearly supervised healthcare environment.

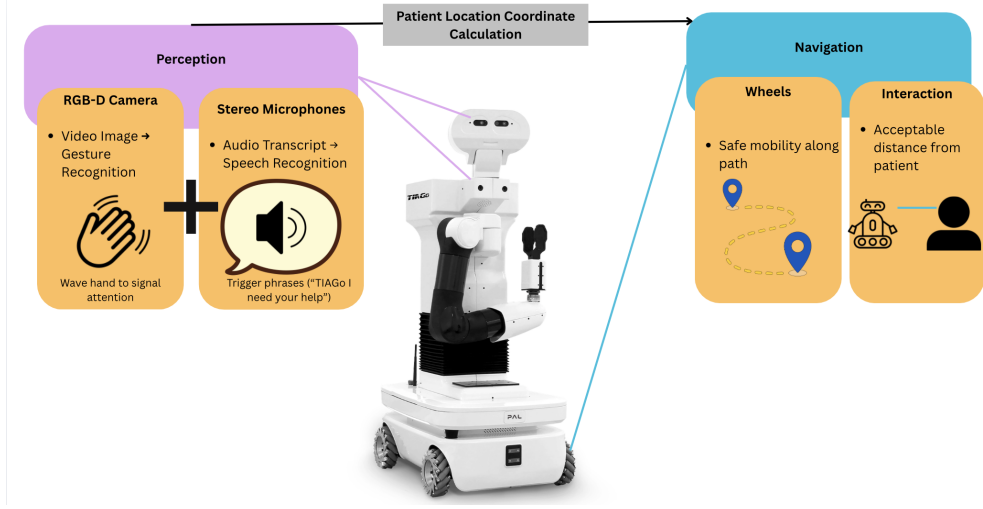


Figure 2: General System Architecture

The system was developed and validated within a Webots-based simulation environment rather than deployed directly onto a physical TIAGo unit. This simulation-first approach enabled controlled, repeatable testing of perception algorithms, navigation strategies, and interaction logic across multiple ward configurations without risking unsafe behaviour in a clinical setting. The TIAGo platform was selected for its established use in domestic and assistive-care research, its compatibility with ROS2 middleware, and the availability of simulation-ready models through the HuNavSim containerised framework.

1.3 Requirements and Design Considerations

Table 1: Key requirements and corresponding design decisions.

Requirement	Explanation	Design Influence
Patient safety	Operation in a clinical environment with vulnerable users	Conservative navigation (low speed, obstacle avoidance) and restriction to low-risk behaviours
User acceptability	Interaction must be comfortable and non-intrusive	Human-centred design with controlled approach distance and predefined responses
Reliable request detection	Accurate identification of user intent	Multimodal perception (speech + gesture) to reduce false positives
Autonomous navigation	Independent movement to reduce staff workload	ROS2 Nav2 stack with SLAM, AMCL, and path planning
System modularity	Ease of modification and extensibility	Separation into perception and navigation sub-systems
Simulation-based validation	Safe and repeatable testing	Development in Webots with controlled environments

1.4 Summary

This project presents a socially assistive robotic system addressing the cumulative burden of low-acuity patient requests on chemotherapy ward nursing staff. The TIAGo-based system uses staged multimodal perception for engagement detection, safe corridor-based navigation, and a bounded interact-and-defer model that escalates beyond its scope. While validated in simulation, the work demonstrates that assistive healthcare robotics can be developed under realistic constraints when ethical boundaries and clinical workflow integration are treated as foundational design requirements.

TIAGo-Based Assistive Robot System for Chemotherapy Wards: A Technical Analysis

Abstract—This paper presents the technical analysis of a socially assistive TIAGo-based robotic system for low-acuity support in chemotherapy wards. The system combines staged multimodal perception (speech-triggered activation followed by temporal gesture confirmation), ROS2-based Nav2 navigation, and a bounded navigate-and-attend interaction model, developed using Webots simulation informed by clinical observation at St Thomas’ Hospital. Evaluation across $n = 10$ trials achieved a 92% end-to-end success rate, 94% navigation success rate, and 97.6% gesture recognition accuracy. However, all testing was conducted in simulation under simplified conditions, and hardware integration with the physical TIAGo platform was not achieved. The work represents a quantitatively supported proof-of-concept rather than a deployment-ready system.

I. INTRODUCTION AND BACKGROUND

The integration of socially assistive robotics into healthcare environments presents an opportunity to support clinical workflows while improving patient experience. Chemotherapy wards are particularly suitable for such systems, as patients often remain seated for extended periods and require intermittent, low-acuity assistance. These requests such as asking for water, blankets, or basic information are individually simple but collectively contribute to increased workload for nursing staff. An assistive robotic system can help offload these repetitive tasks, allowing clinicians to focus on higher-priority medical responsibilities, a role increasingly recognised for assistive robotics in healthcare contexts [2].

To ensure that the system design was grounded in real clinical practice, a representative ward layout was developed following a site visit to St Thomas’ Hospital, where discussions were held with nursing staff regarding typical workflows, spatial constraints, and patient needs. These insights informed the structure of the environment, ensuring that the proposed system reflects realistic operational conditions rather than an idealised setting.

However, deploying robots in this context introduces significant constraints. The environment is spatially structured but sensitive, requiring safe navigation, predictable behaviour, and minimal disruption to both patients and staff. Unlike industrial settings, the robot must operate within a human-centred space where comfort, dignity, and trust are critical [12].

The resulting ward layout, shown in Fig. 1, includes a central corridor for navigation, treatment bays on either side, a nurse station, and a waiting area. This environment directly informs key system design decisions, particularly in relation to navigation safety, engagement detection across multiple viewpoints, and maintaining appropriate proximity to patients.

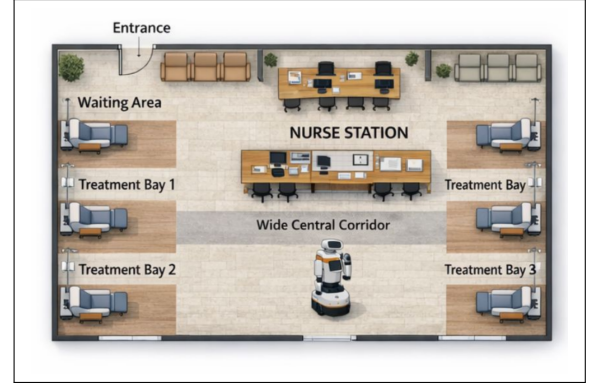


Fig. 1: Chemotherapy ward layout developed based on observations from St Thomas’ Hospital and discussions with clinical staff.

II. SYSTEM ARCHITECTURE

The software architecture is built on the Robot Operating System 2 (ROS2) middleware framework, which provides distributed communication between components through topics, services, and actions [1], [8]. The system is organised into three modular subsystems, *perception*, *navigation*, and *interaction*, which together implement the core product logic: detecting a request for assistance, moving toward the patient safely, and responding through predefined low-acuity support behaviours.

A modular ROS2 architecture was selected for two reasons. First, it supports parallel development, which was important given that perception, navigation, and interaction were developed by different team members. Second, it enables subsystem-level testing when full end-to-end deployment is not yet possible. This was especially valuable once hardware integration became unreliable in the later stages of the project.

However, modularity also introduced technical costs. Topic-based communication simplifies subsystem separation, but it can make timing dependencies harder to manage. In this project, the perception-to-navigation handoff depended on correct sequencing between audio triggering, visual confirmation, target localisation, and navigation goal publication. This means that a failure in one subsystem does not always present as a hard system crash; instead, it may appear as delayed or missing behaviour downstream. As a result, debugging the integrated pipeline required not only algorithm verification but also validation of message flow, synchronisation, and process isolation.

Rather than developing directly on the physical TIAGo unit, a ROS2-based simulation environment was adopted. This allowed the team to model perception algorithms, navigation strategies, and interaction logic within a controlled ward-representative setting without risking unsafe behaviour in a clinical environment [1], [2]. Simulation-first development is also consistent with standard robotics practice where repeated testing, rapid iteration, and safe failure handling are required before physical deployment. Attempts were made to deploy the

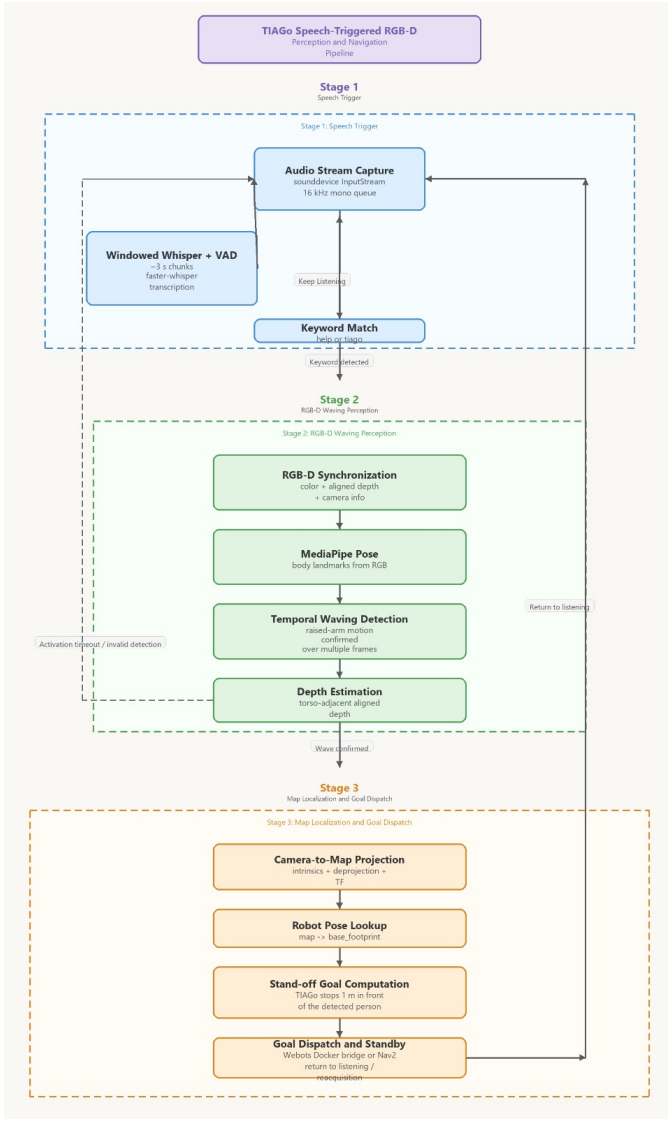


Fig. 2: High-level system architecture showing the multimodal perception pipeline, confirmation stage, and navigation goal generation process.

system onto a physical TIAGo unit later in development, but full integration on hardware was not achieved.

III. DEPLOYMENT ENVIRONMENT

A critical early step was establishing the spatial and operational context of the intended deployment. Chemotherapy wards are semi-structured indoor environments: they are more predictable than open public spaces, yet more constrained and safety-sensitive than generic laboratory test areas. This had direct consequences for both perception and navigation design.

The proposed deployment environment represents a mid-sized chemotherapy treatment ward consisting of a central corridor, treatment bays on either side, a nurse station, a waiting area, and a single entrance/exit route. Each treatment bay contains chairs, IV stands, and associated equipment. This layout matters technically because the robot is not navigating through an empty corridor; it is navigating through a partially occupied and clinically active space in which both static and

dynamic obstacles may restrict path quality and approach behaviour.

A. Implications for System Design

The central corridor provides the most reliable global navigation route and therefore becomes the natural default traversal zone for the robot. This simplifies global path planning and reduces the likelihood of entering cluttered bay regions unnecessarily. By contrast, the treatment bays are obstacle-dense local spaces, requiring finer short-range control and conservative stopping behaviour. The system therefore adopts a patient-approach policy in which the robot navigates toward a patient-adjacent stopping position rather than directly to the user's body location.

This layout also shaped perception requirements. Because engagement may originate from either side of the corridor, the robot must detect visual gestures across bilateral viewing angles and respond to audio cues in a reverberant indoor environment. In practical terms, this motivates a wide-field visual perception strategy and robust audio pre-filtering rather than naive continuous full transcription.

A key consideration is the presence of clinically active zones, such as the nurse station, which serves as a central coordination point within the ward. Navigation through this area must be carefully managed, as paths that are geometrically efficient may still disrupt critical clinical workflows.

IV. SIMULATION ENVIRONMENT

A. Requirements

The simulation environment was required to satisfy four core conditions:

- 1) represent a TIAGo-like mobile robot with usable sensors and controllable kinematics;
- 2) model a ward-like indoor environment with corridors, bays, and furniture;
- 3) include impassable or avoidance-relevant obstacles such as walls, chairs, and IV stands;
- 4) support dynamic human movement to test local obstacle avoidance and recovery behaviour.

These requirements were not only visual or illustrative. They were chosen so that simulation could meaningfully support subsystem testing rather than simply act as a demonstration backdrop.

B. Initial Resources and Compatibility Challenges

The team first investigated official TIAGo tutorial resources through the ROS documentation portal [3]. These resources were useful for understanding expected workflows, but they were largely ROS1-centred and therefore misaligned with a ROS2-first development architecture [4]. This was a major inconvenience as adopting a legacy tutorial path would have increased integration debt later by forcing a migration between middleware paradigms.

The official ROS2 simulation repository published by PAL Robotics was then investigated [3]. Although functionally closer to the target architecture, it imposed a hardware and operating-system requirement that proved difficult for the team to satisfy consistently. Apple Silicon systems, x86-specific virtual machine dependencies, and poor Gazebo performance

on available machines meant that even technically valid simulation options were not practically sustainable for collaborative development.

This became an important project-management issue disguised as a technical issue. A simulator would not be useful if it can only one team member can run it reliably. Therefore, platform selection had to balance not only fidelity, but also accessibility, reproducibility, and stability across the team’s actual hardware.

C. Container-Based Simulation with HuNavSim

To address cross-platform compatibility, the team explored containerised simulation environments using Docker [5]. Containerisation offered two advantages. First, it reduced setup variance across machines. Secondly, the focus shifted from deploying a full robotics stack natively onto each machine to identifying a solution that could be executed reliably to support consistent and repeatable testing.

The HuNavSim framework [14] was identified as a promising option because it provided ROS2-compatible container workflows for several simulators relevant to human–robot navigation:

- 1) **Gazebo Classic** — ROS2 navigation stack with optional world-generation support.
- 2) **Gazebo Fortress** — newer Gazebo version, but without a ready-to-use TIAGo workflow at evaluation time.
- 3) **NVIDIA Isaac Sim** — high-fidelity simulation requiring a dedicated NVIDIA GPU [6].
- 4) **Webots** — practical mobile-robot simulation with accessible hardware demands and TIAGo Lite availability [7].

D. Platform Evaluation and Selection

Table I summarises the main decision criteria used during platform selection. The evaluation intentionally includes both technical and practical criteria for a balance between high-fidelity simulation and team-wide execution feasibility.

TABLE I: Comparative evaluation of simulation platforms.

Platform	ROS2	TIAGo	GPU	Selected
Gazebo Classic	Yes	Partial	No	No
Gazebo Fortress	Yes	No	No	No
NVIDIA Isaac Sim	Yes	No	Yes	No
Webots	Yes	Yes	No	Yes

Gazebo Classic benefited from maturity and strong ecosystem familiarity, but it was approaching end-of-life and did not offer a stable team-wide TIAGo workflow under the chosen setup. **Gazebo Fortress** offered a more modern stack, but the lack of an immediately usable TIAGo model made it too costly to integrate within the project timeframe. **NVIDIA Isaac Sim** offered the highest realism, but this advantage was irrelevant in practice because no team member had appropriate GPU hardware.

Webots was therefore selected, not because it was the most realistic platform in absolute terms, but because it provided the best trade-off between fidelity, accessibility, ROS2 compatibility, and development speed. This choice should be understood as a constrained engineering decision rather than a claim that Webots is universally superior. It improved team productivity and enabled repeatable testing, but it also introduced a fidelity trade-off: the simulated sensing, dynamics, and human

behaviour are notably simpler than real world conditions. That limitation remained throughout the project and is important when interpreting the final testing results.

V. PERCEPTION SUBSYSTEM

A. Architecture

The perception subsystem adopts a multimodal design combining audio and vision to infer user intent. This reduces false activations compared to single-modality systems: background speech alone should not trigger navigation, and isolated gestures without contextual confirmation should not initiate approach behaviour. In the implemented system, engagement confirmation follows a staged pipeline in which a speech trigger is first detected, followed by visual verification of a gesture.

This behaviour is directly reflected in the runtime implementation (Appendix E). Speech-based keyword detection (Listing 1) activates the visual confirmation stage, which is executed as part of the main perception pipeline. This staged design was selected to reduce false positives, limit computational overhead, and satisfy the ethical constraint that the robot should not approach users unnecessarily.

It is important to note that although early feasibility work explored YOLOv8-based gesture detection, the final integrated system instead uses MediaPipe pose estimation with custom temporal wave detection. The subsystem described here therefore reflects the implemented pipeline rather than the initial proposal-stage design.

B. Audio Perception

Table II summarises the core functions of the audio perception module.

TABLE II: Audio perception module functions.

Function	Role
<code>__init__()</code>	Configures the Faster-Whisper model, detects hardware mode, sets precision, and initialises the streaming audio buffer.
<code>audio_callback()</code>	Captures incoming microphone audio and appends it to the running buffer.
<code>process_audio()</code>	Extracts a fixed audio window, applies VAD, performs transcription, and returns text for trigger detection.

The audio pipeline operates using a buffered streaming approach, where incoming samples are accumulated and processed in fixed-length windows of approximately 3 s. As shown in Listing 1, transcription is performed using Faster-Whisper with Voice Activity Detection (VAD) to suppress silent segments. The resulting text is then passed to a keyword-matching stage to detect engagement cues.

This approach is computationally efficient but introduces latency, since inference depends on window accumulation rather than continuous processing. The design therefore trades responsiveness for stability and feasibility on limited hardware. For low-acuity assistance tasks, this latency is acceptable, although it would limit performance in more conversational interaction scenarios.

A further limitation is robustness to environmental noise. While VAD and keyword filtering improve selectivity, the

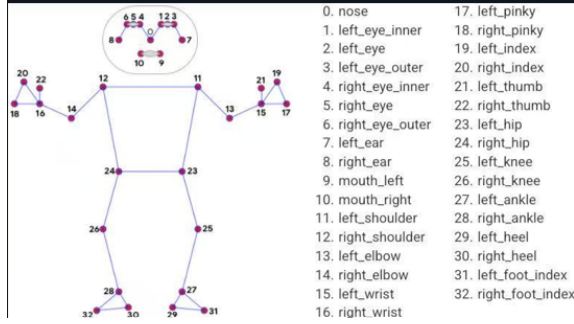


Fig. 3: The 33-point MediaPipe body keypoint model used for gesture recognition.

system was not quantitatively evaluated across diverse acoustic conditions such as overlapping speech or accent variability, and this remains an open limitation.

C. Vision (Gesture) Perception

The visual perception pipeline uses MediaPipe pose estimation combined with a custom temporal wave detector. This provides real-time performance while maintaining a lightweight computational footprint.

Table III summarises the main functions of the wave-detection module.

TABLE III: Wave detection module functions.

Function	Role
<code>__init__()</code>	Defines motion thresholds and initialises history buffers for wrist tracking.
<code>reset()</code>	Clears motion history when valid pose data is lost.
<code>update()</code>	Updates motion state and determines waving status from temporal wrist movement.
<code>_arm_signal()</code>	Computes wrist displacement relative to the shoulder and checks raised-arm geometry.
<code>_compute_bbox()</code>	Builds a body bounding box from visible keypoints.
<code>_compute_torso_pixel()</code>	Estimates torso centre for target localisation.
<code>_depth_sample_pixels()</code>	Selects depth-sampling keypoints for RGB-D estimation.
<code>_visible()</code>	Filters unreliable keypoints using confidence thresholds.
<code>_to_pixel()</code>	Converts normalised coordinates to image-space pixels.

Gesture detection is implemented as a temporal process (Listing 3), where waving is defined as sustained lateral wrist motion over multiple frames subject to posture constraints. This temporal filtering reduces false positives from incidental movements.

Once a waving gesture is confirmed, the system estimates the user's position using RGB-D depth sampling (Listing 5) and projects this into 3D space using camera intrinsics (Listing 6). The use of multiple sampling windows and median filtering improves robustness to noisy depth measurements.

While this approach is efficient and interpretable, it is sensitive to occlusion, partial visibility, and pose-estimation errors. In realistic ward conditions, patients may be seated, partially covered, or asymmetrically positioned, all of which can degrade detection reliability. The method is therefore appropriate for proof-of-concept validation but requires further robustness improvements for deployment.



Fig. 4: Example of waving detection with pose overlay and bounding box.

D. Body Pose Module

Table IV summarises the body-pose controller responsible for camera handling and perception integration.

TABLE IV: Body pose module functions.

Function	Role
<code>_try_open_capture()</code>	Opens a camera device connection.
<code>_resolve_camera_candidates()</code>	Builds a candidate list for robust device selection.
<code>_prime_capture()</code>	Warms up the camera and verifies frame continuity.
<code>_open_camera()</code>	Selects and opens the first valid device.
<code>_read_frame()</code>	Retrieves frames with fault tolerance.
<code>_enable_preview_window()</code>	Creates an OpenCV visualisation window.
<code>_ensure_preview_window()</code>	Maintains the preview window.
<code>body_key()</code>	Coordinates frame capture, pose estimation, and wave detection.

Although relatively simple, this module is critical for system stability. In practice, perception failures often arise from device initialisation or frame acquisition issues rather than model performance. Explicit handling of camera setup and shutdown improves robustness of the overall pipeline.

E. Multimodal Integration Pipeline

Table V summarises the top-level integration logic.

TABLE V: Multimodal integration pipeline functions.

Function	Role
<code>run_camera_subprocess()</code>	Launches gesture detection in a separate subprocess.
<code>main_pipeline()</code>	Coordinates audio triggering, visual confirmation, and goal generation.

The system begins in a passive listening state. Upon detecting a valid speech trigger (Listing 1), the visual detection stage is activated. A confirmed waving gesture then produces a target location, which is converted into a navigation goal (Listing 7) and dispatched through the perception-to-navigation callback.

This staged design improves robustness by isolating computationally expensive visual processing and reducing resource contention. However, it introduces sequential dependencies between perception stages, meaning delays in audio or vision propagate through the system. The architecture therefore prioritises reliability over fully concurrent interaction, which is

appropriate for a proof-of-concept system but would require redesign for real-time deployment.

VI. NAVIGATION SUBSYSTEM

The ROS2 Navigation Stack (Nav2) was selected for its maturity and wide adoption in mobile robotics [8]. The navigation pipeline used the standard layered structure of mapping, localisation, costmap generation, global planning, local control, and behaviour management. Webots provided the simulated ward environment, while RViz supported observation of map, pose, and planning state. The pipeline can be demonstrated as:

Sensors → SLAM → Map → AMCL → Costmap
→ Planner → Controller → Motion

A. Mapping and Localisation

The first stage was generating a spatial representation of the environment and estimating the robot's pose within it. LiDAR and RGB-D sensing were used to construct an occupancy-style map of pathways and obstacles. Manual teleoperation was used to gather sufficient environmental coverage, which was a pragmatic choice given the limited time available for autonomous exploration tuning.

Adaptive Monte Carlo Localisation (AMCL) was then used to estimate the robot pose relative to the prebuilt map [9]. Once perception confirmed a patient request, the estimated user location was converted into a navigation goal and published to the Nav2 action server.

This mapping and localisation strategy is appropriate for a semi-structured ward, but it relies on the assumption that the environment remains close enough to the mapped configuration for localisation to remain stable. In a real chemotherapy ward, moved furniture, people clustering, or equipment repositioning could challenge that assumption more than in simulation.

B. Costmap Construction

The Nav2 costmap layer converted the static map and live sensor updates into a traversability representation. The inflation layer expanded obstacles by a safety margin based on the robot footprint, preventing the planner from generating routes that were geometrically valid but clinically unsafe.

This distinction was important as a robot can avoid collisions mathematically while still passing uncomfortably close to beds, chairs, or IV stands. The inflation-based buffer therefore plays both a geometric and a social role in this project.

C. Path Planning

Global path planning used NavFn, which implements a Dijkstra-based search over the costmap to generate a route to the target location. In the ward setting, this worked well because the navigation space is relatively structured, with corridor-like connectivity and a limited number of meaningful approach routes.

The main limitation is that optimality in the planner is defined relative to the costmap, not to clinical workflow. The shortest path may not always be the least disruptive path. This reinforces the earlier point that healthcare navigation quality should not be assessed only by geometric efficiency.



Fig. 5: SLAM mapping process within the Webots environment.

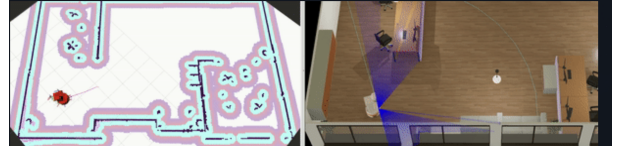


Fig. 6: Generated 2D grid map of the simulated ward environment.

D. Local Control and Obstacle Avoidance

The DWB local planner adjusted the trajectory in real time by sampling candidate velocity commands and selecting the one that best balanced path following and obstacle avoidance. This was suitable for handling moving obstacles such as simulated staff or patient movement in the corridor.

The strength of this approach is responsiveness. The weakness is sensitivity to tuning: overly conservative parameters may cause hesitation or oscillation, while overly aggressive parameters may reduce social comfort even if collisions are avoided. Therefore local navigation performance is treated as functionally acceptable in simulation, but not fully optimised.

E. Behaviour Logic

Nav2 behaviour trees coordinated higher-level navigation decisions, including replanning, recovery behaviours, and fall-back to standby states. This was valuable for a project of this type because it provided recoverability without requiring all failure modes to be handled manually in custom code.

From a critical perspective, the use of behaviour trees strengthened the navigation architecture by making recovery logic explicit. However, the integrated system would still benefit from higher-level human-aware policies, such as more deliberate yielding near patients or staff, which were outside the final project scope.

VII. INTERACTION MODULE

The interaction module coordinates the robot's response once the perception subsystem has confirmed patient engagement. In the implemented system, interaction takes the form of a navigate-and-attend sequence: upon receiving the confirmed multimodal trigger, the robot navigates toward the detected user and maintains a predefined stand-off distance, signalling acknowledgement through physical presence rather than explicit dialogue.

This minimal interaction model was a deliberate scope decision. Research in socially intelligent robotics has shown that patients are more receptive when interactions are clearly bounded, predictable, and non-intrusive [10]. In a chemotherapy ward, where patients are physically vulnerable and may be fatigued, an overly conversational robot risks causing discomfort or confusion. The navigate-and-attend approach provides a predictable, bounded response that clinical staff can easily interpret and override, consistent with established principles for safe physical human-robot interaction [12]. It also avoids

delivering incorrect verbal responses in a setting where the consequences of miscommunication are higher than in general-purpose assistive contexts.

Where a request exceeds the robot’s capabilities, the system defers to nursing staff rather than attempting an independent response. This escalation boundary ensures the robot operates within its functional scope and does not assume clinical responsibility. The system must also communicate clearly that it is an automated assistant, as confusion about competence can lead to misplaced trust [13].

Future work would extend this layer to include structured dialogue, task selection from a predefined assistance taxonomy, and patient feedback mechanisms [2].

VIII. TESTING PROTOCOLS

A. Simulation-Based Validation

Testing was designed to evaluate the reliability of the integrated perception–navigation pipeline within a simulated ward environment. All experiments were conducted in Webots using a representative layout containing static obstacles (e.g., equipment and furniture) and dynamic agents representing patients and staff. Reported values correspond to mean performance across $n = 10$ trials per condition.

Four test categories were defined to capture both navigation and interaction performance.

Navigation accuracy tests measured the Euclidean distance between the robot’s final pose and the assigned goal across multiple treatment-bay targets and randomised starting positions.

Obstacle avoidance tests evaluated whether the robot could reach goals without collision under both static and dynamic conditions, while logging recovery and replanning events.

Stopping distance tests assessed whether the robot maintained a socially appropriate stand-off distance from the patient, based on the target offset defined in the navigation controller.

Localisation stability tests monitored AMCL pose estimation during extended traversals to identify drift or relocalisation instability.

In addition, perception performance was evaluated through detection accuracy, latency, and localisation error, reflecting the system’s dependence on reliable engagement detection prior to navigation.

Metrics were obtained from ROS2 logging outputs, including navigation feedback, pose estimation data, and perception module outputs. Post-processing scripts were used to compute averages over repeated trials.

B. Results Summary

A condensed summary of key performance metrics is shown in Table VI, with the full set of results provided in Appendix A (Table IX).

The system achieved an end-to-end success rate of 92%, conditioned on valid perception triggers, and a navigation success rate of 94%. The difference reflects failure cases primarily arising from perception-stage limitations rather than navigation errors.

Navigation performance was consistent, with a mean time-to-goal of 41.7 s and path efficiency of 87.5%, indicating that

TABLE VI: Condensed summary of key system performance metrics.

Metric	Value	Unit
End-to-end success rate	92.0	%
Navigation success rate	94.0	%
Time to goal	41.7	s
Final position error	0.12	m
Stopping distance error	0.18	m
Path efficiency	87.5	%
Detection latency	185	ms
Gesture recognition accuracy	97.6	%

the planner produced near-optimal trajectories while accounting for obstacle avoidance and safety constraints. Final position and stopping errors remained within acceptable bounds (0.12 m and 0.18 m respectively), demonstrating reliable convergence to target regions while maintaining a safe interaction distance.

Perception performance showed a balanced trade-off between precision (92.6%) and recall (89.8%), resulting in an F1-score of 91.2% (see Appendix A). Gesture recognition accuracy was high (97.6%), while detection latency averaged 185 ms, supporting near real-time responsiveness. Target localisation error (0.21 m) contributed directly to navigation goal uncertainty.

Overall, the results indicate that the system demonstrates stable and reliable behaviour in simulation, with performance primarily constrained by perception robustness rather than navigation capability.

C. Observed Limitations

Despite the positive results, several limitations remain. First, all testing was conducted in simulation, meaning that sensor noise, lighting variation, and human motion were simplified relative to real-world conditions. Second, system evaluation was conditioned on successful perception triggers, which limits the ability to fully quantify end-to-end reliability under realistic interaction scenarios. Third, results are reported as mean values without detailed statistical analysis, and future work should incorporate variance and confidence intervals to better characterise robustness.

D. Planned Physical Testing

Future work will extend the current evaluation framework to the physical TIAGo platform. This will involve replicating the simulation test categories while incorporating real-world factors such as sensor noise, human variability, and environmental unpredictability. In particular, physical testing will assess the transferability of simulation-tuned parameters, including stopping distance thresholds, obstacle inflation margins, and gesture detection reliability, as well as user comfort and workflow impact.

IX. PROJECT MANAGEMENT

A. Project Plan and Timeline

The project followed a Scrum-based structure with stand-ups, shared Trello tracking, and separate work packages for perception, navigation, interaction, integration, and documentation. The scope centred on engagement detection, safe navigation, and bounded verbal assistance, with tray delivery

retained only as a stretch goal. The complete timeline is presented in Appendix E.

A pre-study phase (mid-September to mid-November 2025) covered project scoping, literature review, YOLOv8 feasibility testing, deployment environment analysis informed by the St Thomas' Hospital visit, and simulation platform evaluation. Perception and navigation then proceeded in parallel from mid-November through mid-January 2026, since neither had a blocking dependency on the other. Audio perception and interaction were developed from early December alongside both streams. This parallel structure ensured all subsystems were individually functional before integration began in mid-February.

B. Deviations and Lessons Learned

The most disruptive deviation was hardware failure. From late February, the physical TIAGo became increasingly unreliable: intermittent SSH connectivity loss, failure to publish ROS2 topics after boot, and persistent inability to restore the RGB camera feed. These issues consumed the time reserved for hardware integration, and the two-week integration window left no recovery margin.

The critical decision came in mid-March, ten days before Demo Day. After three weeks of troubleshooting with no consistent improvement, the team committed fully to simulation-based delivery, continuing would have risked arriving with neither a reliable physical demonstration nor a polished simulation pipeline.

Consequently, the project scope was deliberately narrowed from the originally proposed broader assistive-care concept to a complete end-to-end simulation pipeline. This was a managed trade-off: reducing scope allowed the team to produce a quantitatively evaluated system rather than a partially integrated one. Team members originally allocated to hardware were reassigned to testing, documentation, and demonstration preparation.

Three lessons emerged. First, hardware reliability risk must be formally mitigated from the outset, not assumed away. Second, simulation should be a formal fallback delivery route from the beginning, the pivot was only viable because Webots had been sufficiently developed during the core phase. Third, a minimum two-week buffer before delivery is essential for absorbing unpredictable late-stage issues.

X. STAKEHOLDER ANALYSIS

The deployment of socially assistive robots in healthcare involves multiple stakeholders whose interests are aligned only partially.

Patients are the primary intended beneficiaries. For this group, the central issues are comfort, dignity, predictability, and trust. A technically impressive system that behaves intrusively or unpredictably would fail even if its navigation and perception performance were acceptable [10].

Nursing and clinical staff are secondary operational stakeholders. They may benefit from reduced interruption for minor non-clinical tasks, but only if the system is genuinely reliable and does not create a new supervisory burden. Therefore, staff acceptance depends as much on trust and escalation behaviour as on nominal efficiency.

Hospital administrators are concerned with compliance, procurement justification, risk, and cost-benefit trade-offs.

Their perspective is less about whether the robot can function technically and more about whether it can be governed safely and maintained sustainably.

Regulatory and ethics bodies require assurance that the system stays within appropriate safety, data, and liability boundaries.

Future developers and researchers are long-term stakeholders because the modular system architecture is intended to support later extension, validation, and redesign.

A critical point is that stakeholder success criteria are not identical. A design that looks efficient from an administrative perspective may still be unsatisfactory from a patient dignity perspective. The system therefore has to be judged against more than one axis of performance.

XI. SUSTAINABILITY AND ETHICS ANALYSIS

A. Ethical Overview

In a chemotherapy ward, ethics is not an external layer added after technical design; it is part of the design constraint set. Privacy, dignity, autonomy, and safety shape what the robot is allowed to do, how it approaches, and what it should never attempt to automate.

Table VII presents an indicative ethical decision matrix comparing TIAGo-assisted and human-only handling of low-acuity requests. This matrix should be interpreted as a structured design reflection, not as empirical proof from a clinical trial.

TABLE VII: Indicative ethical decision matrix for TIAGo deployment. Scores are scenario-based, not clinically validated.

Criterion	Wt.	TIAGo	Human	Justification
Safety	.25	4	3	Reduced workload-related error risk
Efficiency	.20	5	3	Faster low-acuity response potential
Experience	.15	4	5	Human empathy remains stronger
Privacy	.15	3	4	Sensor use creates GDPR concern
Staff load	.15	5	2	Repetitive-task reduction
Reliability	.10	3	4	Humans remain more adaptive
Total	1.0	4.15	3.45	

The value of this matrix is not the exact final score, but the fact that it exposes trade-offs. TIAGo may improve efficiency and staff load, but these gains do not remove privacy and reliability concerns.

B. Patient Privacy and Data Protection

The perception subsystem processes continuous visual and audio input in a setting where privacy expectations are high. The system therefore assumes local processing wherever feasible, avoids storing raw data beyond the immediate interaction cycle, and treats gesture detection as behavioural classification rather than identity recognition. These choices reduce risk, but they do not eliminate it. Residual privacy concerns remain because the robot is still sensing in a vulnerable environment and could be perceived as surveillance if not introduced transparently [11].

C. Patient Dignity and Autonomy

The robot is intentionally limited to low-acuity assistance and uses a conservative social standoff distance rather than close physical approach. Patients can ignore or dismiss the interaction without penalty, and the robot does not attempt

emotionally manipulative or overly human-like dialogue. This is ethically important because in a chemotherapy ward, preserving a patient's sense of control is at least as important as reducing wait time.

D. Safety and Liability

Physical safety is supported through low speed, conservative stopping behaviour, and obstacle-aware navigation [12]. Functional safety is supported by scope restriction: the robot does not deliver medication, provide clinical advice, or take decisions reserved for staff. The escalation pathway to nursing staff is therefore both a safety mechanism and a liability boundary.

E. Bias and Fairness in Perception

Gesture and audio perception may perform unevenly across body types, mobility states, accents, and speech clarity profiles. This is particularly important in chemotherapy contexts, where patients may be fatigued, seated in unusual postures, or speaking weakly. The system therefore cannot assume fairness simply because the algorithms function in controlled tests. Continuous evaluation across diverse user conditions would be necessary before claiming equitable performance.

F. Ethical Dilemmas

Table VIII summarises the main design tensions.

TABLE VIII: Key ethical dilemmas in TIAGo deployment.

Dilemma	Values	Risk	Mitigation
Automation vs. care	Efficiency vs. empathy	Reduced human interaction	Hybrid model
Data vs. privacy	Performance vs. GDPR	Data misuse	Local processing
Innovation vs. safety	Technical ambition vs. reliability	System failure	Fail-safe design
Cost vs. access	Innovation vs. equity	Unequal access	Scalable deployment strategy

These dilemmas reinforce that success is not defined only by technical capability. A clinically acceptable robot must remain limited, interpretable, and governable.

G. Informed Consent and Transparency

Patients should be informed of the robot's capabilities, limitations, and sensing behaviour as part of the care environment. The system must also clearly communicate that it is an automated assistant rather than a clinical authority [13]. This is especially important because confusion about competence can lead to misplaced trust.

H. Sustainability Considerations

Simulation-first development reduces hardware wear, repeated setup overhead, and unnecessary energy usage during early iteration. More broadly, if assistive robotics reduces repetitive interruption of nursing staff, it may contribute indirectly to more efficient staff utilisation. However, sustainability claims should remain modest. A single robot does not transform healthcare environmental performance on its own; any meaningful sustainability argument depends on deployment scale, maintenance demands, and long-term operational efficiency.

XII. CONCLUSION

This project presented the design and implementation of a socially assistive robotic system based on the TIAGo platform, targeting low-acuity support within chemotherapy ward environments. The final system integrates multimodal perception and ROS2-based navigation within a simulation-first development framework, demonstrating a complete perception-to-navigation pipeline.

From a technical perspective, the work establishes the feasibility of constructing a modular assistive-care system under realistic development constraints. Key strengths include the staged multimodal perception approach, which reduces false activations, the use of a robust ROS2 navigation stack, and the development of a structured simulation environment for controlled evaluation. Quantitative results from simulation-based testing (summarised in Section VIII and detailed in Appendix C) demonstrate consistent navigation performance, high task success rates, and acceptable localisation and stopping accuracy within the defined environment.

However, important limitations remain. While quantitative metrics were obtained, these are restricted to simulation conditions and do not fully capture real-world variability in sensing, human behaviour, and environmental complexity. In particular, perception robustness under occlusion, acoustic noise, and diverse user conditions was not comprehensively validated. Furthermore, hardware integration challenges with the physical TIAGo platform prevented full end-to-end testing on the real system.

Overall, the project is best characterised as a quantitatively supported proof-of-concept rather than a deployment-ready system. Future work should focus on transferring the validated pipeline to the physical robot, expanding evaluation across more realistic and varied conditions, and strengthening robustness of the perception subsystem for real clinical environments.

REFERENCES

- [1] M. Quigley *et al.*, "ROS: an open-source Robot Operating System," in *Proc. ICRA Workshop Open Source Softw.*, 2009.
- [2] B. Siciliano and O. Khatib, Eds., *Springer Handbook of Robotics*, 2nd ed. Springer, 2016.
- [3] PAL Robotics, "TIAGo simulation," GitHub, 2024. [Online]. Available: https://github.com/pal-robotics/tiago_simulation
- [4] Open Robotics, "ROS 2 Design," 2024. [Online]. Available: <https://design.ros2.org/>
- [5] D. Merkel, "Docker: Lightweight Linux containers for consistent development and deployment," *Linux J.*, vol. 2014, no. 239, 2014.
- [6] NVIDIA, "Isaac Sim," 2024. [Online]. Available: <https://developer.nvidia.com/isaac-sim>
- [7] O. Michel, "Webots: Professional mobile robot simulation," *Int. J. Adv. Robot. Syst.*, vol. 1, no. 1, pp. 39–42, 2004.
- [8] S. Macenski *et al.*, "Robot Operating System 2: Design, architecture, and uses in the wild," *Sci. Robot.*, vol. 7, no. 66, 2022.
- [9] S. Thrun, W. Burgard, and D. Fox, *Probabilistic Robotics*. MIT Press, 2005.
- [10] K. Dautenhahn, "Socially intelligent robots: Dimensions of human–robot interaction," *Phil. Trans. R. Soc. B*, vol. 362, no. 1480, pp. 679–704, 2007.
- [11] Information Commissioner's Office, "UK GDPR," 2023. [Online]. Available: <https://ico.org.uk/>
- [12] S. Haddadin and E. Croft, "Physical human–robot interaction," in *Springer Handbook of Robotics*, B. Siciliano and O. Khatib, Eds. Springer, 2016, pp. 1835–1874.
- [13] J. Bryson and A. Winfield, "Standardizing ethical design for AI and autonomous systems," *Computer*, vol. 50, no. 5, pp. 116–119, 2017.
- [14] Lixiangqi2002, "HuNavSim Docker containers," 2024. [Online]. Available: https://github.com/Lixiangqi2002/hunavsim_docker

B. B. System Deployment Context



Fig. 7: TIAGo robot platform used during development within the project space.

This figure shows the physical TIAGo unit used during development attempts prior to transitioning fully to simulation-based validation.

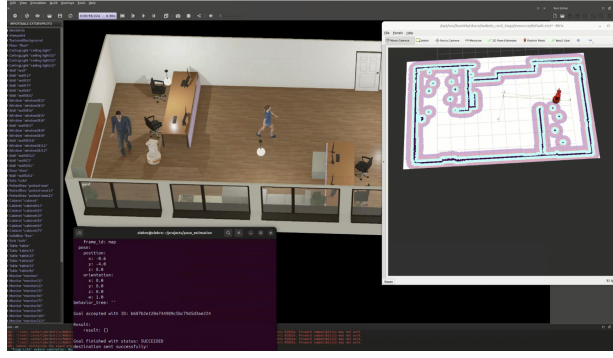


Fig. 8: Webots Simulation

This figure shows the working Webots simulation running an example test scenario. The internal mapping is also shown as a separate window.

C. C. Full Testing Metrics

The following metrics represent mean performance across $n = 10$ trials conducted in a simulated chemotherapy ward environment with both static and dynamic obstacles. A condensed subset of these results is presented in the main text (Section X).

TABLE IX: Complete system testing metrics (see Section X for summary).

Metric	Definition	Unit	Value
End-to-End System Metrics			
Full task completion time	Total time to complete full perception–navigation cycle	s	74.3
End-to-end task success rate	Successful trials given valid perception trigger	%	92.0
Manual intervention count	Human interventions required per trial	/trial	0.2
Stopping distance error	Deviation from desired stand-off distance	m	0.18
Navigation Metrics			
Navigation success rate	Trials reaching goal region without failure	%	94.0
Time to goal	Time taken to reach navigation target	s	41.7
Path length	Total distance travelled by robot	m	8.6
Path efficiency	Ratio of optimal to executed path	%	87.5
Final position error	Distance between goal and final robot pose	m	0.12
Final orientation error	Angular deviation at goal	deg	7.4
Collision count	Minor contacts per trial	/trial	0.1
Recovery events	Replanning or recovery behaviours triggered	/trial	0.3
Perception Metrics			
Detection precision	Correct detections among predicted positives	%	92.6
Detection recall	Detected targets among all true targets	%	89.8
F1-score	Harmonic mean of precision and recall	%	91.2
False positive rate	Incorrect detections among negatives	%	5.1
False negative rate	Missed detections among true targets	%	10.2
Gesture accuracy	Correct classification of waving gestures	%	97.6
Gesture confirmation latency	Delay from appearance to detection output	ms	185
Target localisation error	Error in estimated human position	m	0.21

D. D. Key Code Snippets from the Final Runtime Pipeline

This section presents selected excerpts from the final implemented perception–navigation pipeline. The aim is not to reproduce the full source code, but to highlight the main logic used in the current runtime path: speech-triggered activation, temporal waving detection, RGB-D depth estimation, map-frame goal generation, and navigation dispatch. These listings are referenced throughout the Technical Analysis section to connect the system description to the implemented code.

1) *Speech Trigger and Keyword Matching*: The first stage of the pipeline normalises incoming transcripts and checks for configured trigger words such as `help` and `tiago`. This provides a lightweight gating mechanism before activating the more expensive RGB-D perception stage.

```
_NON_ALNUM_PATTERN = re.compile(r"^[^a-z0-9]+")

def normalize_text(text: str) -> str:
    normalized = _NON_ALNUM_PATTERN.sub(" ", (text or "").lower())
    return " ".join(normalized.split())
```

```

def find_keywords(text: str, keywords: Sequence[str]) -> tuple[str, ...]:
    normalized_text = normalize_text(text)
    if not normalized_text:
        return tuple()

    tokens = set(normalized_text.split())
    matches: list[str] = []

    for keyword in keywords:
        normalized_keyword = normalize_text(keyword)
        if not normalized_keyword:
            continue

        if " " in normalized_keyword:
            if normalized_keyword in normalized_text:
                matches.append(normalized_keyword)
        elif normalized_keyword in tokens:
            matches.append(normalized_keyword)

    return tuple(dict.fromkeys(matches))

```

Listing 1: Keyword normalisation and matching from `audio_trigger.py`

2) *Windowed Speech Processing:* The audio subsystem buffers microphone input, forms a fixed transcription window, and applies Faster-Whisper with Voice Activity Detection. A keyword match then activates the visual detection stage.

```

def poll(self) -> Optional[KeywordMatch]:
    if self._model is None:
        raise RuntimeError("Audio keyword trigger has not been started.")

    buffered_chunks = []
    while not self._audio_queue.empty():
        buffered_chunks.append(self._audio_queue.get().flatten())

    if buffered_chunks:
        self._audio_buffer = np.concatenate([self._audio_buffer, *buffered_chunks])

    if len(self._audio_buffer) < self._block_size:
        return None

    chunk = self._audio_buffer[: self._block_size]
    self._audio_buffer = self._audio_buffer[self._block_size :]

    segments, _ = self._model.transcribe(
        chunk,
        language="en",
        vad_filter=True,
        vad_parameters={"min_silence_duration_ms": 500},
    )

    for segment in segments:
        transcript = segment.text.strip()
        matched_keywords = find_keywords(transcript, self._keywords)
        if matched_keywords:
            return KeywordMatch(transcript=transcript, keywords=matched_keywords)

    return None

```

Listing 2: Windowed speech trigger from `audio_trigger.py`

3) *Temporal Waving Detection:* Once the visual stage is active, the waving detector confirms a gesture only when horizontal wrist motion is sustained over multiple frames, helping reject transient or accidental movements.

```

def update(
    self,
    landmarks: Sequence[object],
    image_width: int,
    image_height: int,
) -> WaveDetection:
    if not landmarks:
        self.reset()
        return WaveDetection(False, None, None, [], None)

    bbox = _compute_bbox(
        landmarks,
        image_width,
        image_height,
        self._visibility_threshold,
    )

```

```

for arm in ("left", "right"):
    signal = _arm_signal(landmarks, arm, self._visibility_threshold, self._raise_margin)
    if signal is None:
        self._histories[arm].clear()
        continue
    self._histories[arm].append(signal)

winning_arm = None
for arm, history in self._histories.items():
    if len(history) >= self._min_wave_samples:
        if max(history) - min(history) >= self._motion_threshold:
            winning_arm = arm
            break

return WaveDetection(
    is_waving=winning_arm is not None,
    arm=winning_arm,
    target_pixel=_compute_torso_pixel(landmarks, image_width, image_height),
    depth_sample_pixels=_depth_sample_pixels(landmarks, image_width, image_height),
    bbox=bbox,
)

```

Listing 3: Main waving decision logic from `wave_detection.py`

4) *Arm Motion Heuristic:* A valid arm signal is only accepted when the wrist is raised above the shoulder and the elbow is not dropped. This reduces false positives from unrelated arm movement.

```

def _arm_signal(
    landmarks: Sequence[object],
    arm: str,
    visibility_threshold: float,
    raise_margin: float,
) -> Optional[float]:
    shoulder_idx = LEFT_SHOULDER if arm == "left" else RIGHT_SHOULDER
    elbow_idx = LEFT_ELBOW if arm == "left" else RIGHT_ELBOW
    wrist_idx = LEFT_WRIST if arm == "left" else RIGHT_WRIST

    shoulder = landmarks[shoulder_idx]
    elbow = landmarks[elbow_idx]
    wrist = landmarks[wrist_idx]

    if not all(_visible(point, visibility_threshold) for point in (shoulder, elbow, wrist)):
        return None

    wrist_above_shoulder = wrist.y < (shoulder.y - raise_margin)
    elbow_not_dropped = elbow.y < (shoulder.y + raise_margin)
    if not wrist_above_shoulder or not elbow_not_dropped:
        return None

    return float(wrist.x - shoulder.x)

```

Listing 4: Raised-arm motion heuristic from `wave_detection.py`

5) *Depth Estimation and 3D Projection:* After a waving person is confirmed, the system estimates depth using multiple torso-adjacent pixels rather than a single sample. The detected torso location is then projected from image space into the 3D camera frame.

```

def median_depth_from_windows(
    depth_image: np.ndarray,
    sample_pixels: Iterable[Tuple[int, int]],
    encoding: str,
    window_radius: int,
    min_depth_m: float,
    max_depth_m: float,
) -> Optional[float]:
    if depth_image.ndim != 2:
        raise ValueError("Depth image must be a single-channel array")

    scale = depth_unit_scale_from_encoding(encoding)
    height, width = depth_image.shape
    collected = []

    for u, v in sample_pixels:
        if u < 0 or v < 0 or u >= width or v >= height:
            continue
        u_min = max(0, u - window_radius)
        u_max = min(width, u + window_radius + 1)
        v_min = max(0, v - window_radius)

```

```

    v_max = min(height, v + window_radius + 1)
    patch = depth_image[v_min:v_max, u_min:u_max].astype(np.float32) * scale
    patch = patch[np.isfinite(patch)]
    patch = patch[(patch >= min_depth_m) & (patch <= max_depth_m)]
    if patch.size:
        collected.append(patch)

    if not collected:
        return None

    stacked = np.concatenate(collected)
    return float(np.median(stacked))

```

Listing 5: Robust depth estimation from geometry.py

```

def deproject_pixel_to_camera(
    pixel_u: int,
    pixel_v: int,
    depth_m: float,
    intrinsics: CameraIntrinsics,
) -> Tuple[float, float, float]:
    if depth_m <= 0.0:
        raise ValueError("Depth must be positive")

    x = (float(pixel_u) - intrinsics.cx) * depth_m / intrinsics.fx
    y = (float(pixel_v) - intrinsics.cy) * depth_m / intrinsics.fy
    z = depth_m
    return x, y, z

```

Listing 6: Pixel deprojection from geometry.py

6) *Stand-Off Goal Computation:* The navigation target is not the patient position itself. Instead, the robot stops one metre short of the detected person while remaining oriented toward them.

```

def compute_standoff_goal(
    robot_xy: Sequence[float],
    person_xy: Sequence[float],
    stand_off_distance_m: float,
) -> Pose2D:
    robot_x, robot_y = float(robot_xy[0]), float(robot_xy[1])
    person_x, person_y = float(person_xy[0]), float(person_xy[1])

    dx = person_x - robot_x
    dy = person_y - robot_y
    distance = math.hypot(dx, dy)
    facing_yaw = math.atan2(dy, dx)

    if distance < 1e-6:
        return Pose2D(x=robot_x, y=robot_y, yaw=facing_yaw)

    approach_distance = max(distance - stand_off_distance_m, 0.0)
    scale = approach_distance / distance
    goal_x = robot_x + dx * scale
    goal_y = robot_y + dy * scale
    return Pose2D(x=goal_x, y=goal_y, yaw=facing_yaw)

```

Listing 7: Stand-off goal computation from geometry.py

7) *End-to-End RGB-D Callback:* The main ROS2 callback synchronises RGB and depth inputs, confirms waving, estimates depth, projects the detected person into the map frame, and prepares a runtime navigation goal.

```

def _rgbd_callback(
    self,
    color_msg: Image,
    depth_msg: Image,
    camera_info_msg: CameraInfo,
) -> None:
    self._expire_activation_if_needed()
    if self._audio_enabled and not self._vision_active:
        return

    if self._target_locked and not self._should_reacquire():
        return

    color_bgr = self._bridge.imgmsg_to_cv2(color_msg, desired_encoding="bgr8")
    depth_image = self._bridge.imgmsg_to_cv2(depth_msg, desired_encoding="passthrough")
    color_rgb = cv2.cvtColor(color_bgr, cv2.COLOR_BGR2RGB)

    results = self._pose_estimator.process(color_rgb)

```

```

if not results.pose_landmarks:
    self._wave_detector.reset()
    self._publish_debug(color_bgr, color_msg.header)
    return

image_height, image_width = color_bgr.shape[:2]
detection = self._wave_detector.update(
    results.pose_landmarks.landmark,
    image_width,
    image_height,
)

if not detection.is_waving or detection.target_pixel is None:
    self._publish_debug(color_bgr, color_msg.header)
    return

depth_m = median_depth_from_windows(
    depth_image=depth_image,
    sample_pixels=detection.depth_sample_pixels or [detection.target_pixel],
    encoding=depth_msg.encoding,
    window_radius=self._depth_window_radius,
    min_depth_m=self._min_depth_m,
    max_depth_m=self._max_depth_m,
)
if depth_m is None:
    self.get_logger().warning("Detected waving person, but depth was invalid.")
    self._publish_debug(color_bgr, color_msg.header)
    return

```

Listing 8: Main perception callback from waving_person_navigator.py

8) *Goal Construction and Dispatch:* Once both the detected person and the robot are localised in the map frame, the node computes the stand-off goal, publishes the detected target and goal poses, and dispatches the navigation command.

```

goal_pose_2d = compute_standoff_goal(
    robot_xy=(robot_pose.pose.position.x, robot_pose.pose.position.y),
    person_xy=(person_point_msg.point.x, person_point_msg.point.y),
    stand_off_distance_m=self._stand_off_distance_m,
)
goal_pose = GoalPose(
    frame_id=self._map_frame,
    x=goal_pose_2d.x,
    y=goal_pose_2d.y,
    yaw=goal_pose_2d.yaw,
)
goal_pose_msg = self._build_goal_pose(goal_pose)

self._person_point_pub.publish(person_point_msg)
self._person_pose_pub.publish(self._build_person_pose(person_point_msg))
self._goal_pose_pub.publish(goal_pose_msg)

self._target_locked = True
self._last_goal_time = self.get_clock().now()

goal_sent = True
if self._auto_send_goal:
    goal_sent = self._dispatch_goal(goal_pose_msg, goal_pose)

```

Listing 9: Goal generation and dispatch from waving_person_navigator.py

9) *Webots Goal Bridge:* For the Webots-based workflow, the runtime goal is converted into a Nav2 action payload and passed into the simulation container through Docker.

```

def build_nav2_goal_payload(goal: GoalPose) -> str:
    qx, qy, qz, qw = quaternion_from_yaw(goal.yaw)
    return (
        "{pose: {header: {frame_id: \"%s\"}, pose: {position: {x: %.6f, y: %.6f, z: 0.0}, "
        "orientation: {x: %.6f, y: %.6f, z: %.6f, w: %.6f}}}}}"
        % (goal.frame_id, goal.x, goal.y, qx, qy, qz, qw)
    )

def send_nav2_goal(
    goal: GoalPose | None = None,
    config_path: str | Path = DEFAULT_CONFIG_PATH,
    config: Dict[str, Any] | None = None,
) -> subprocess.CompletedProcess[str]:
    if config is None:
        config = load_goal_config(config_path)

    command = build_docker_nav2_command(config=config, goal=goal)

```

```
return subprocess.run(command, check=True, text=True, capture_output=True)
```

Listing 10: Dynamic Webots goal bridge from `webots_nav_goal.py`

E. E. Project Management

This section provides a visual summary of the project schedule and major work packages undertaken from October 2025 to April 2026. The Gantt chart highlights the sequencing of pre-study work, subsystem development, integration attempts, the transition to simulation-based validation, and final reporting activities.

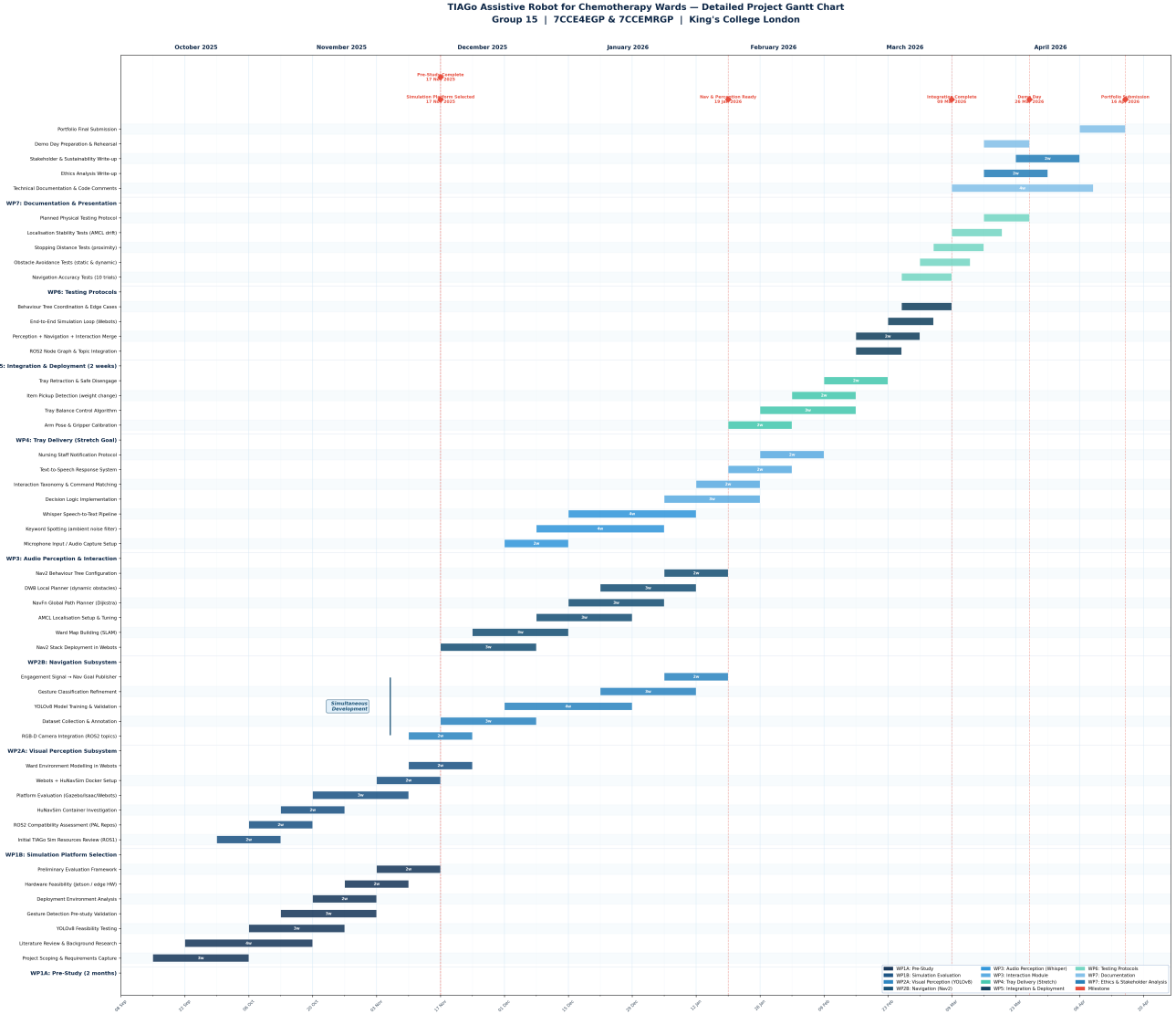


Fig. 9: Project Gantt chart showing the main development phases, integration attempts, and final delivery stages.

The chart illustrates that perception, navigation, and interaction-related work progressed in parallel during the core development period. It also shows the late-stage deviation caused by hardware issues with the physical TIAGo platform, which compressed integration, testing, and final documentation into the final weeks of the project.

F. F. Bill of Materials

The following bill of materials (BOM) represents the software and computational resources required to develop and evaluate the system in a simulation-based environment.

TABLE X: Simulation Bill of Materials

Component	Description	Cost (£)	Dependencies
Webots Simulation Environment	Mobile robot simulator with TIAGo support	0	ROS 2 interface, graphics drivers
ROS 2 (Humble)	Robotics middleware for communication and system integration	0	Ubuntu 22.04, Python 3
Nav2 Stack	Navigation framework including planning and control	0	ROS 2, TF2, sensor topics
SLAM Toolbox / Cartographer	Mapping and localisation algorithms	0	ROS 2, LiDAR, odometry
RViz	Visualisation tool for mapping and navigation	0	ROS 2, TF transforms
MediaPipe	Pose estimation for gesture detection	0	Python, OpenCV
Faster-Whisper	Speech recognition for audio triggering	0	Python, PyTorch
OpenCV	Image processing and camera interfacing	0	Python
Docker / HuNavSim	Containerised simulation and deployment environment	0	Docker engine, Linux kernel
Development Workstations	Personal computers used for simulation and testing	–	CPU, RAM, operating system
Total Cost		0	